

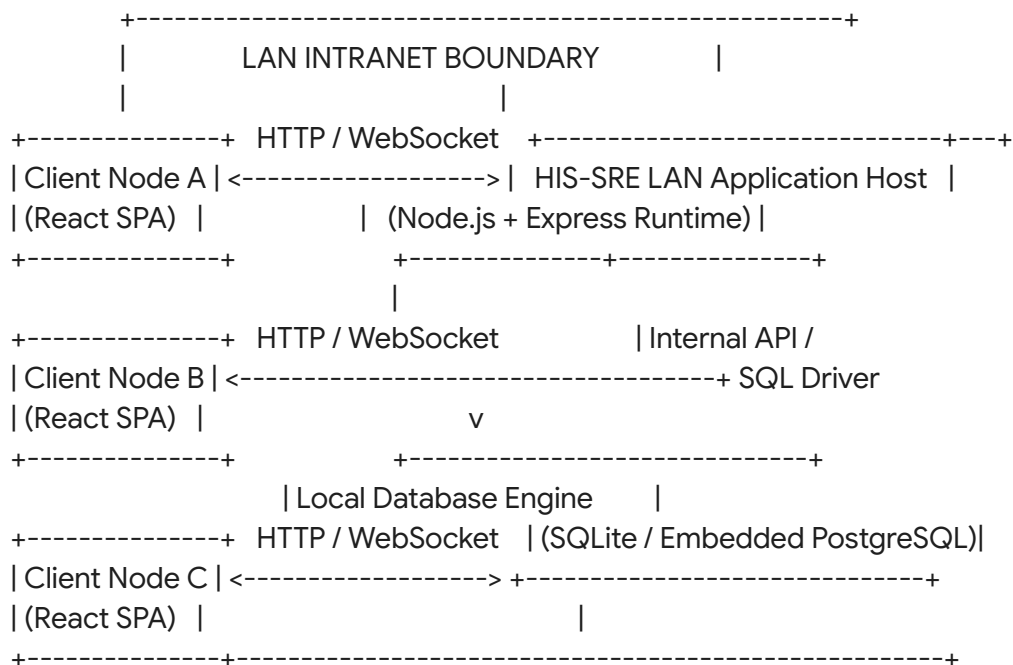
Technical Specification & Systems Architecture Blueprint: HIS University Statistical Recommendation Engine (HIS-SRE)

Section 1: Executive Summary & System Architecture

1.1 System Overview & Deployment Topology

The HIS University Statistical Recommendation Engine (HIS-SRE) is a self-contained, high-performance decision-support system designed to compute specialty fit scores for prospective higher education students. The engine processes high school baccalaureate (BAC) academic performance records alongside 6-dimensional psychometric personality vectors (RIASEC model) to yield a continuous, deterministic suitability percentage (\$0.00\% - 100.00\%\$) across all academic programs offered by HIS University.

To guarantee zero internet dependency, low operational latency, and complete data privacy, the system is architected exclusively for **Local Area Network (LAN) / Intranet** deployment.



1.2 Data Lifecycle & Processing Pipeline

The data processing lifecycle operates across five isolated runtime stages:

[1. Input Ingestion] -> [2. Schema Validation] -> [3. Calculation Engine] -> [4. Local Persistence] -> [5. Analytics Dashboard]

1. **Input Ingestion:** Client captures candidate BAC stream designation, individual subject grades (\$0.00 - 20.00\$), and 6D RIASEC scale responses (\$1 - 5\$ scale).
2. **Validation Layer:** Edge validation via runtime type guards and schema parsing (Zod) ensures numeric bounds, complete subject sets, and vector non-nullability prior to calculation execution.
3. **Calculation Engine:** Vectorized numeric execution evaluating Academic Weighted Performance (S_{academic}) and Cosine Psychometric Similarity (S_{riasec}), returning a combined score (S_{final}).
4. **Local DB Persistence:** Evaluation records, input parameters, and computed match profiles are written asynchronously to the local relational database for audit trailing and institutional analytics.
5. **Dashboard Rendering:** The frontend consumes the structured JSON response payload to render ranked match hierarchies, radar visualizers, and contribution breakdowns.

Section 2: Mathematical & Statistical Engine

2.1 Academic Score Derivation (S_{academic})

Let K be the set of academic subjects required for evaluation. For a student with a verified set of grades $g_k \in [0, 20]$ where $k \in K$, and a specialty i defining a set of subject-specific weighting coefficients $w_{k,i} \in \mathbb{R}_{\geq 0}$, the raw weighted academic score is derived as:

$$S_{\text{academic, raw}, i} = \frac{\sum_{k \in K} w_{k,i} \cdot g_k}{\sum_{k \in K} w_{k,i} \cdot \max(g_k)}$$

Where $\max(g_k) = 20.00$. Converting this to a percentage scale $[0, 100]$:

$$S_{\text{academic, base}, i} = \left(\frac{\sum_{k \in K} w_{k,i} \cdot g_k}{20 \cdot \sum_{k \in K} w_{k,i}} \right) \times 100$$

Stream Compatibility Coefficient ($\mu_{s,i}$)

To account for high school stream alignment (e.g., *Mathematics*, *Experimental Sciences*, *Technical Math*, *Management & Economy*, *Foreign Languages*, *Literature & Philosophy*) without applying hard binary disqualifications, a continuous stream modifier matrix $\mu_{s,i} \in [0.70, 1.00]$ is applied:

$$S_{\text{academic}, i} = S_{\text{academic, base}, i} \cdot \mu_{s,i}$$

2.2 RIASEC Psychometric Vector Similarity (S_{riasec})

Candidate psychometric profiles and specialty benchmark targets are modeled as vectors in a 6-dimensional Hilbert space $\mathbb{R}^6$, corresponding to the Holland Occupational Themes:

$$\vec{V} = \begin{bmatrix} R & I & A & S & E & C \end{bmatrix}^T$$

Let $\vec{u} \in \mathbb{R}^6$ represent the student's normalized RIASEC score vector, and $\vec{v}_i \in \mathbb{R}^6$ represent the benchmark profile vector for specialty i .

6D Cosine Similarity Formula

The unscaled psychometric alignment is computed via the normalized dot product (Cosine Similarity):

$$\text{CosSim}(\vec{u}, \vec{v}_i) = \frac{\vec{u} \cdot \vec{v}_i}{\|\vec{u}\|_2 \|\vec{v}_i\|_2} = \frac{\sum_{m=1}^6 u_m v_{m,i}}{\sqrt{\sum_{m=1}^6 u_m^2} \sqrt{\sum_{m=1}^6 v_{m,i}^2}}$$

Given that all RIASEC input dimensions are non-negative ($u_m, v_{m,i} \geq 0$), $\text{CosSim}(\vec{u}, \vec{v}_i) \in [0.0, 1.0]$. The normalized psychometric score percentage is expressed as:

$$S_{\text{riasec}, i} = \text{CosSim}(\vec{u}, \vec{v}_i) \times 100$$

2.3 Combined Score Integration

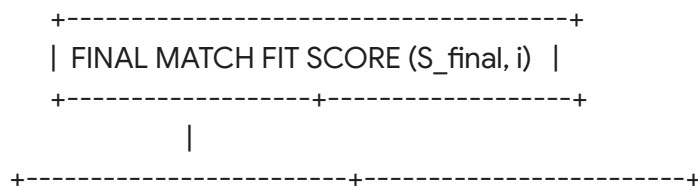
The final specialty fit score $S_{\text{final}, i}$ combines the academic and psychometric dimensions using a 70/30 weighted linear combination:

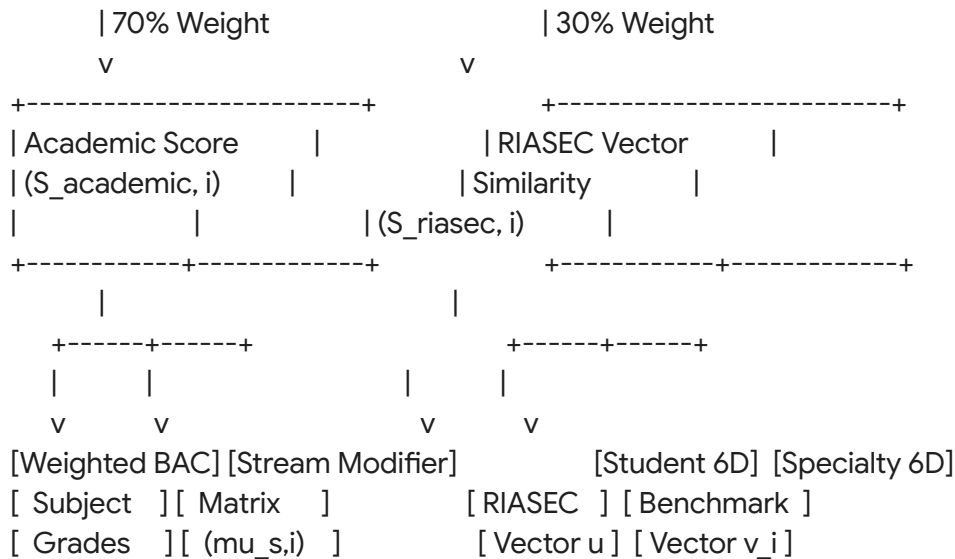
$$S_{\text{final}, i} = (\alpha \cdot S_{\text{academic}, i}) + (\beta \cdot S_{\text{riasec}, i})$$

Where:

- $\alpha = 0.70$ (Academic weighting coefficient)
- $\beta = 0.30$ (Psychometric weighting coefficient)
- $\alpha + \beta = 1.00$

$$S_{\text{final}, i} \in [0.00, 100.00]$$





2.4 Edge-Case Calibration & Boundary Conditions

1. Zero Vector Handling ($\|\vec{u}\|_2 = 0$):

If a student submits all zeros for RIASEC ratings, vector magnitude evaluates to zero, causing a division-by-zero undefined behavior. The system catches this state and assigns a uniform fallback vector $\vec{u}_{\text{fallback}} = [1, 1, 1, 1, 1, 1]^T$, yielding a baseline neutral similarity $\text{CosSim}(\vec{u}_{\text{fallback}}, \vec{v}_i) = 0.50$ (\$50.0\%).

2. Missing BAC Subject Grades:

If non-mandatory elective grades are omitted, the denominator calculation dynamically recalculates over the subset of present subjects $K_{\text{present}} \subseteq K$:

$$S_{\text{academic, base}, i} = \left(\frac{\sum_{k \in K_{\text{present}}} w_{k,i} \cdot g_k}{\sum_{k \in K_{\text{present}}} w_{k,i}} \right) \times 100$$

Section 3: Data Schemas & TypeScript Interfaces

```
// =====
// Core Domain Models & Input Definitions
// =====

export type BacStream =
  | 'MATHEMATICS'
  | 'EXPERIMENTAL_SCIENCES'
  | 'TECHNICAL_MATHEMATICS'
  | 'MANAGEMENT_ECONOMY'
  | 'FOREIGN_LANGUAGES'
```

```

| 'LITERATURE_PHILOSOPHY';

export type SubjectCode =
| 'MATH'
| 'PHYSICS'
| 'NATURAL_SCIENCES'
| 'PHILOSOPHY'
| 'ARABIC'
| 'FRENCH'
| 'ENGLISH'
| 'ACCOUNTING_FINANCE'
| 'ECONOMICS'
| 'HISTORY_GEOGRAPHY';

export interface BacGrades {
  grades: Record<SubjectCode, number>; // Grade value between 0.00 and 20.00
  overallBacMark: number; // Official BAC overall mark
}

export interface RiasecVector {
  realistic: number; // R dimension [1 - 5 scale or aggregate rating]
  investigative: number; // I dimension
  artistic: number; // A dimension
  social: number; // S dimension
  enterprising: number; // E dimension
  conventional: number; // C dimension
}

export interface StudentProfile {
  studentId?: string;
  fullName: string;
  bacStream: BacStream;
  academicPerformance: BacGrades;
  psychometricProfile: RiasecVector;
  evaluatedAt?: Date;
}

// =====
// HIS Specialty Configuration Definitions
// =====

export interface SubjectWeightMap {
  weights: Partial<Record<SubjectCode, number>>; // Weight coefficient w_{k,i}

```

```
}
```

```
export interface RiasecBenchmark {  
  vector: RiasecVector;  
}
```

```
export interface HisSpecialtyConfig {  
  id: string;  
  code: string;  
  title: string;  
  department: string;  
  description: string;  
  subjectWeights: SubjectWeightMap;  
  streamModifiers: Record<BacStream, number>; // Factor mu_{s,i} in range [0.70, 1.00]  
  riasecBenchmark: RiasecBenchmark;  
  isActive: boolean;  
}
```

```
// =====  
// Engine Output & Evaluation Schema Definitions  
// =====
```

```
export interface SpecialtyMatchBreakdown {  
  specialtyId: string;  
  specialtyCode: string;  
  specialtyTitle: string;  
  department: string;  
  academicScore: number; // S_{academic, i} (0 - 100)  
  psychometricScore: number; // S_{riasec, i} (0 - 100)  
  finalScore: number; // S_{final, i} (0 - 100)  
  rank: number; // Ordinal match position  
  details: {  
    streamModifierApplied: number;  
    rawAcademicPercentage: number;  
    vectorCosineSimilarity: number;  
  };  
}
```

```
export interface CalculationResult {  
  evaluationId: string;  
  timestamp: string;  
  studentName: string;  
  bacStream: BacStream;
```

```

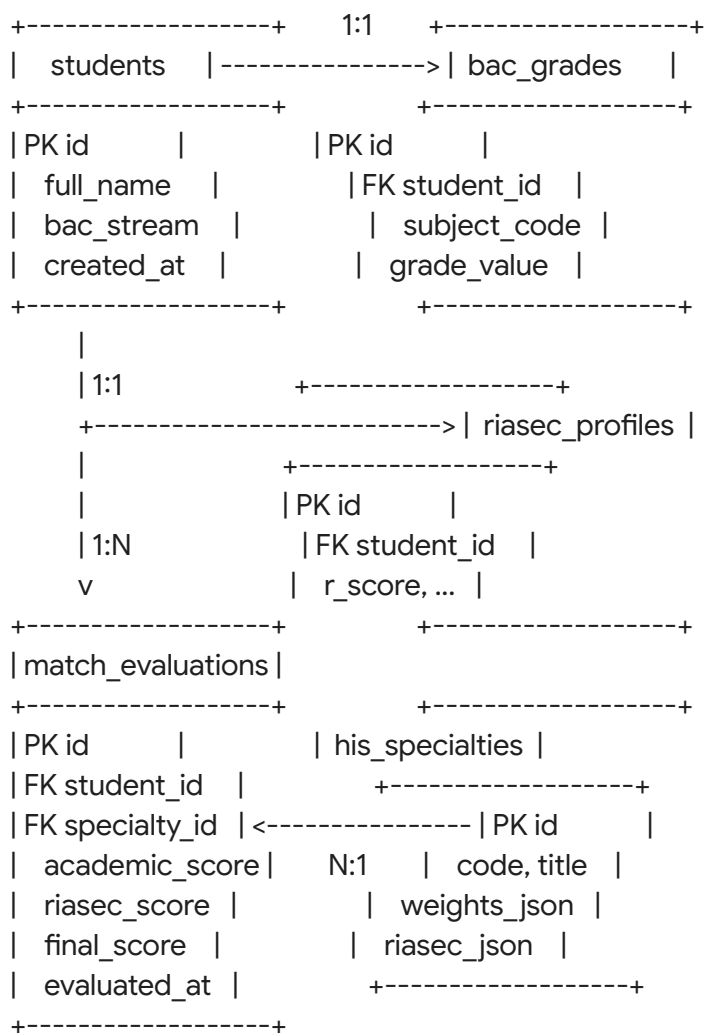
    matches: SpecialtyMatchBreakdown[];
}

```

Section 4: Local Database Design (LAN Focus)

4.1 Relational Entity-Relationship Description

The local relational schema is structured to maintain raw submission parameters alongside match outcomes for audit trails and offline institutional reporting.



4.2 DDL Schema Statements (SQLite / PostgreSQL Compatible)

-- Students Master Table

```
CREATE TABLE IF NOT EXISTS students (  
  id TEXT PRIMARY KEY,  
  full_name VARCHAR(255) NOT NULL,  
  bac_stream VARCHAR(50) NOT NULL,  
  overall_bac_mark NUMERIC(4, 2) NOT NULL CHECK (overall_bac_mark BETWEEN 0.00 AND 20.00),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- Academic Marks Table

```
CREATE TABLE IF NOT EXISTS bac_grades (  
  id TEXT PRIMARY KEY,  
  student_id TEXT NOT NULL REFERENCES students(id) ON DELETE CASCADE,  
  subject_code VARCHAR(50) NOT NULL,  
  grade_value NUMERIC(4, 2) NOT NULL CHECK (grade_value BETWEEN 0.00 AND 20.00),  
  UNIQUE(student_id, subject_code)  
);
```

-- Psychometric Vector Table

```
CREATE TABLE IF NOT EXISTS riasec_profiles (  
  id TEXT PRIMARY KEY,  
  student_id TEXT NOT NULL UNIQUE REFERENCES students(id) ON DELETE CASCADE,  
  realistic_score NUMERIC(3, 2) NOT NULL CHECK (realistic_score > 0.00),  
  investigative_score NUMERIC(3, 2) NOT NULL CHECK (investigative_score > 0.00),  
  artistic_score NUMERIC(3, 2) NOT NULL CHECK (artistic_score > 0.00),  
  social_score NUMERIC(3, 2) NOT NULL CHECK (social_score > 0.00),  
  enterprising_score NUMERIC(3, 2) NOT NULL CHECK (enterprising_score > 0.00),  
  conventional_score NUMERIC(3, 2) NOT NULL CHECK (conventional_score > 0.00)  
);
```

-- HIS Specialties Benchmark Configuration Table

```
CREATE TABLE IF NOT EXISTS his_specialties (  
  id TEXT PRIMARY KEY,  
  code VARCHAR(50) UNIQUE NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  department VARCHAR(100) NOT NULL,  
  description TEXT,  
  subject_weights_json TEXT NOT NULL, -- JSON containing SubjectWeightMap
```



```

stream_modifiers_json TEXT NOT NULL, -- JSON containing BacStream modifier mapping
riasec_benchmark_json TEXT NOT NULL, -- JSON containing RiasecVector benchmark
is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Match Evaluation Results Persistence
CREATE TABLE IF NOT EXISTS match_evaluations (
  id TEXT PRIMARY KEY,
  student_id TEXT NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  specialty_id TEXT NOT NULL REFERENCES his_specialties(id) ON DELETE CASCADE,
  academic_score NUMERIC(5, 2) NOT NULL,
  riasec_score NUMERIC(5, 2) NOT NULL,
  final_score NUMERIC(5, 2) NOT NULL,
  rank_position INTEGER NOT NULL,
  evaluated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for Fast Local Network Query Execution
CREATE INDEX IF NOT EXISTS idx_evaluations_student ON match_evaluations(student_id);
CREATE INDEX IF NOT EXISTS idx_evaluations_specialty ON match_evaluations(specialty_id);
CREATE INDEX IF NOT EXISTS idx_evaluations_final_score ON match_evaluations(final_score
DESC);

```

4.3 Offline Data Backup & Institutional Export Strategy

To facilitate institutional reporting without requiring external databases or cloud connections:

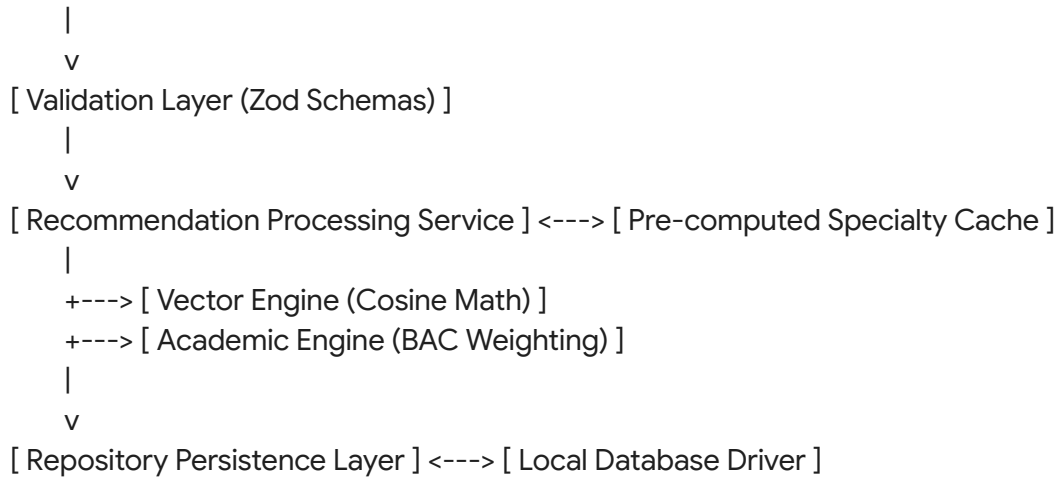
- **SQLite File Snapshotting:** Cron-based local backup script executes hourly, replicating the active .sqlite database file to a local USB key or secondary host network directory.
- **JSON/CSV Serializer Module:** Backend service exposes dedicated data-dump endpoints (/api/v1/export/evaluations?format=csv) yielding flat CSV records formatted for administration software (Excel/SPSS).

Section 5: API & Backend Service Architecture (Node.js + TS)

5.1 Service Layer Topology

The Node.js backend uses a layered architecture separating API HTTP transport from execution logic.

[Express Routing Layer]



5.2 Zod API Payload Validation Schemas

```
import { z } from 'zod';
```

```
export const BacStreamSchema = z.enum([  
  'MATHEMATICS',  
  'EXPERIMENTAL_SCIENCES',  
  'TECHNICAL_MATHEMATICS',  
  'MANAGEMENT_ECONOMY',  
  'FOREIGN_LANGUAGES',  
  'LITERATURE_PHILOSOPHY'  
]);
```

```
export const SubjectCodeSchema = z.enum([  
  'MATH',  
  'PHYSICS',  
  'NATURAL_SCIENCES',  
  'PHILOSOPHY',  
  'ARABIC',  
  'FRENCH',  
  'ENGLISH',  
  'ACCOUNTING_FINANCE',  
  'ECONOMICS',  
  'HISTORY_GEOGRAPHY'  
]);
```

```

export const CalculateRecommendationSchema = z.object({
  fullName: z.string().min(2).max(250),
  bacStream: BacStreamSchema,
  overallBacMark: z.number().min(0.00).max(20.00),
  grades: z.record(SubjectCodeSchema, z.number().min(0.00).max(20.00)),
  riasec: z.object({
    realistic: z.number().min(0.00).max(100.00),
    investigative: z.number().min(0.00).max(100.00),
    artistic: z.number().min(0.00).max(100.00),
    social: z.number().min(0.00).max(100.00),
    enterprising: z.number().min(0.00).max(100.00),
    conventional: z.number().min(0.00).max(100.00),
  })
});

export type CalculateRecommendationInput = z.infer<typeof
CalculateRecommendationSchema>;

```

5.3 Endpoint Contracts

POST /api/v1/recommendations/calculate

- **Description:** Computes specialty match percentages for a candidate profile and persists results locally.
- **Request Headers:** Content-Type: application/json
- **Request Body:** CalculateRecommendationInput
- **Response Status Codes:** 200 OK, 400 Bad Request, 500 Internal Server Error
- **Sample Response Payload:**

```

{
  "evaluationId": "eval_8f9a2b1c-4d3e-4f5a-8b9c-1d2e3f4a5b6c",
  "timestamp": "2026-08-12T10:30:00.000Z",
  "studentName": "Alex Mercer",
  "bacStream": "MATHEMATICS",
  "matches": [
    {
      "specialtyId": "spec_cs_01",
      "specialtyCode": "HIS-CS-AI",
      "specialtyTitle": "B.Sc. Artificial Intelligence & Software Engineering",

```

```

    "department": "Computer Science",
    "academicScore": 92.50,
    "psychometricScore": 88.40,
    "finalScore": 91.27,
    "rank": 1,
    "details": {
      "streamModifierApplied": 1.00,
      "rawAcademicPercentage": 92.50,
      "vectorCosineSimilarity": 0.884
    }
  },
  {
    "specialtyId": "spec_ba_02",
    "specialtyCode": "HIS-BUS-FIN",
    "specialtyTitle": "B.Sc. Financial Technology & Analytics",
    "department": "Business Administration",
    "academicScore": 84.00,
    "psychometricScore": 72.10,
    "finalScore": 80.43,
    "rank": 2,
    "details": {
      "streamModifierApplied": 0.95,
      "rawAcademicPercentage": 88.42,
      "vectorCosineSimilarity": 0.721
    }
  }
]
}

```

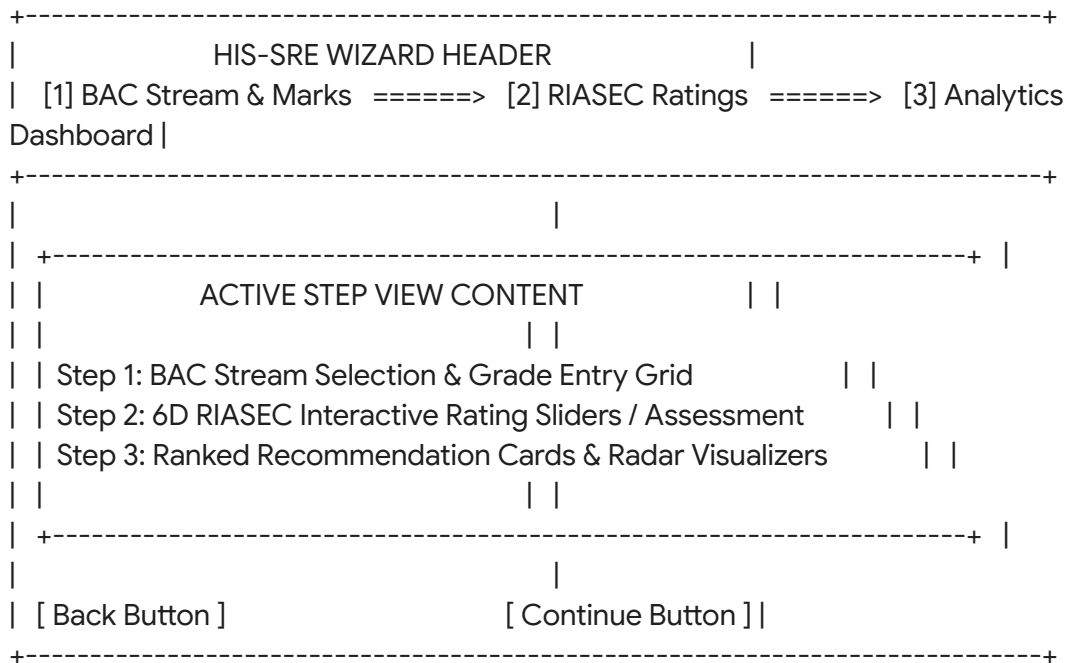
5.4 Low-Latency Performance Optimizations

To achieve real-time response times ($< 15\text{ms}$) on modest local hardware:

1. **Pre-computed Benchmark Magnitudes:** In-memory caching of pre-calculated L_2 norms for specialty benchmark vectors ($\|\vec{v}_i\|_2$), eliminating redundant square root operations during similarity computations.
2. **In-Memory Config Caching:** Active HIS specialty JSON records are loaded into a single Node.js memory map on application startup. Database disk reads are completely bypassed during matrix execution.
3. **Asynchronous Batch DB Persistence:** Computation output is instantly dispatched to the HTTP response payload; database persistence is queued asynchronously via event loop microtasks (`process.nextTick()`).

Section 6: Frontend Design (React + Tailwind CSS)

6.1 Multi-Step Wizard UI Architecture



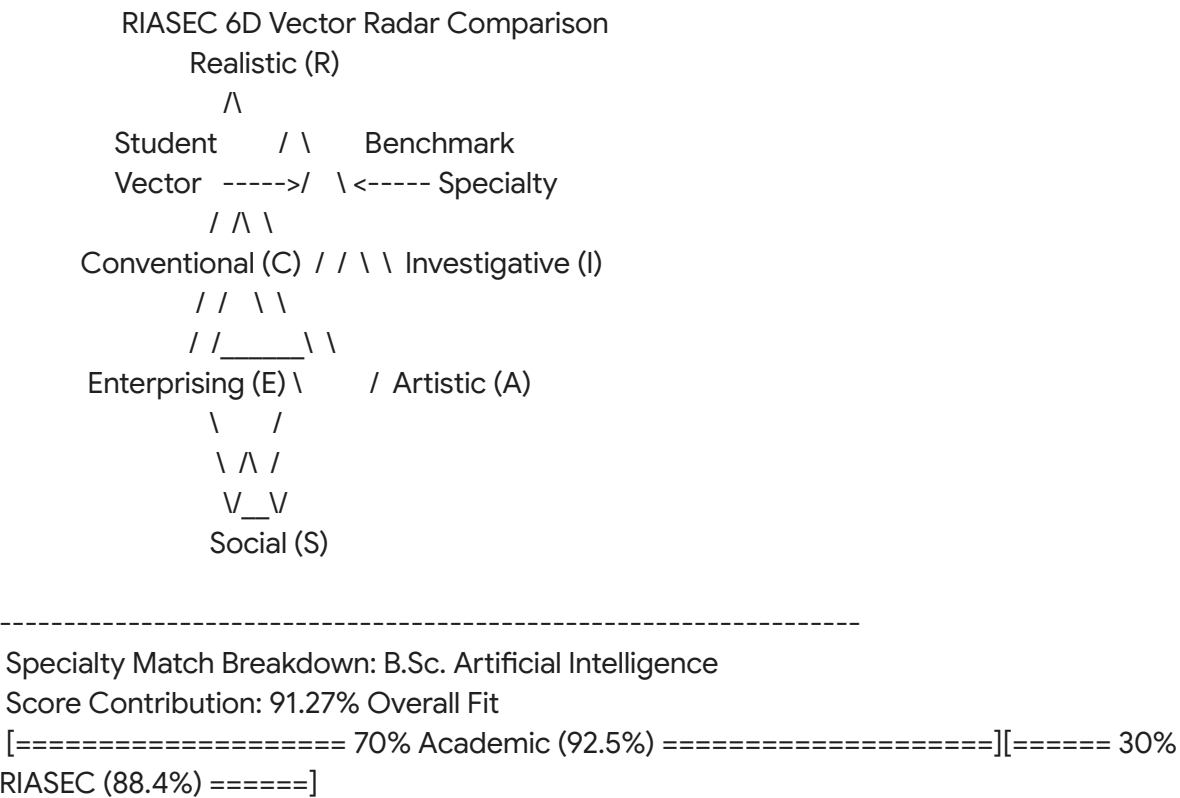
6.2 UI Component Tree

- <WizardLayout>: Root container managing Wizard state machine (\$Step \in \{1, 2, 3\}\$).
 - <WizardProgressBar>: Visual step indicator.
 - <Step1AcademicForm>:
 - <StreamSelector>: Dropdown input for BAC stream selection.
 - <SubjectGradeGrid>: Dynamic input fields bound to stream-specific subjects with validation boundaries (\$0.00 - 20.00\$).
 - <Step2RiasecForm>:
 - <RiasecSliderGroup>: 6 dimension slider interfaces (\$R, I, A, S, E, C\$) with real-time numeric readouts.
 - <Step3AnalyticsDashboard>:
 - <SummaryCardHeader>: Candidate metadata panel.
 - <VisualizationSection>:
 - <RiasecRadarChart>: SVG radar visualizer rendering student vector vs. top recommendation vector.
 - <RankedSpecialtyList>: Ordered list of calculated match results.
 - <SpecialtyMatchCard>: Display card showing final fit percentage badge, rank ordinal, and score breakdown toggles.
 - <ScoreContributionBar>: Dual-color bar showing 70% Academic vs. 30% Psychometric

split.

6.3 Analytics Dashboard Visualizer Specification

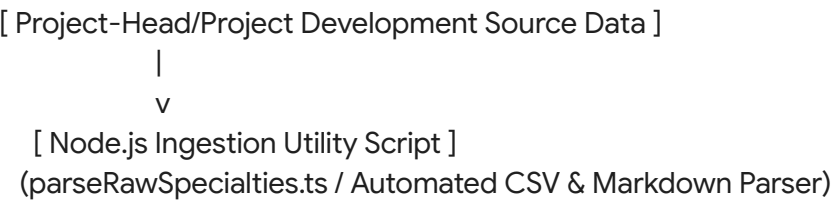
The analytics dashboard uses a combined SVG radar visualization and progress indicator:



Section 7: Dataset Mapping & Ingestion Strategy

7.1 Dataset Parsing Pipeline

To ingest raw institutional domain data from the reference repository (Project-Head/Project Development) into static local database seed files:



```

|
+----> Validates schema structure
+----> Maps raw subjects to SubjectCode enum
+----> Computes normalized baseline RIASEC vectors
|
v
[ JSON Seed Files / Local DB Seeder ]
(specialties.seed.json -> SQLite Persistence)

```

7.2 Database Seeder Blueprint (seedLocalDatabase.ts)

```

import fs from 'fs';
import path from 'path';

export interface RawSpecialtySeed {
  code: string;
  title: string;
  department: string;
  description: string;
  weights: Record<string, number>;
  streamModifiers: Record<string, number>;
  riasecBenchmark: {
    R: number; I: number; A: number; S: number; E: number; C: number;
  };
}

export async function parseAndSeedSpecialties(filePath: string): Promise<void> {
  const absolutePath = path.resolve(filePath);
  const rawData = fs.readFileSync(absolutePath, 'utf-8');
  const parsedSeedData: RawSpecialtySeed[] = JSON.parse(rawData);

  for (const item of parsedSeedData) {
    const formattedSpecialty = {
      id: `spec_${item.code.toLowerCase().replace(/^[a-z0-9]/g, '_')}`,
      code: item.code,
      title: item.title,
      department: item.department,
      description: item.description,

```

```

subjectWeightsJson: JSON.stringify({ weights: item.weights }),
streamModifiersJson: JSON.stringify(item.streamModifiers),
riasecBenchmarkJson: JSON.stringify({
  vector: {
    realistic: item.riasecBenchmark.R,
    investigative: item.riasecBenchmark.I,
    artistic: item.riasecBenchmark.A,
    social: item.riasecBenchmark.S,
    enterprising: item.riasecBenchmark.E,
    conventional: item.riasecBenchmark.C,
  }
}),
isActive: true,
};

// Execute SQLite / PostgreSQL UPSERT query
// INSERT INTO his_specialties (...) VALUES (...) ON CONFLICT(code) DO UPDATE ...
}
}

```

Section 8: Step-by-Step Implementation Roadmap

Execution is structured across four phases to systematically move from mathematical validation to offline LAN deployment.

Phase 1: Algorithm Verification & Mathematical Prototyping

- **Task 1.1:** Build isolated TypeScript unit tests for the mathematical engine using vitest or jest.
- **Task 1.2:** Validate S_{academic} edge cases: missing subject grades, single-grade edge values (\$0.00\$ and \$20.00\$), and stream modifier scale validation.
- **Task 1.3:** Validate S_{riasec} 6D Cosine Vector Math: test orthogonality, identical vectors ($\text{CosSim} = 1.0$), diametric vectors, zero vectors, and boundary values.
- **Task 1.4:** Verify \$70/30\$ weighted score integration outputs across a synthetic benchmark dataset of 100 student profiles.

Phase 2: Core TypeScript Engine & API Implementation

- **Task 2.1:** Initialize Node.js TypeScript project, setting up strict tsconfig.json configurations.
- **Task 2.2:** Construct local database layer (SQLite embedded driver + migration scripts using DDL schema from Section 4).
- **Task 2.3:** Implement data seed parsing pipeline (seedLocalDatabase.ts) to populate his_specialties table from repository raw files.
- **Task 2.4:** Implement RecommendationService containing pre-computed benchmark matrix caching and asynchronous calculation dispatch.

- **Task 2.5:** Setup Express routing layer with Zod validation middleware for POST `/api/v1/recommendations/calculate`.

Phase 3: UI/UX Component Development & Local State Hooks

- **Task 3.1:** Scaffold React SPA using Vite + Tailwind CSS framework.
- **Task 3.2:** Implement Multi-Step Wizard state machine hook (`useRecommendationWizard`).
- **Task 3.3:** Build `Step1AcademicForm` components featuring dynamic subject grid mapping based on chosen BAC stream.
- **Task 3.4:** Build `Step2RiasecForm` slider controls with interactive value readouts.
- **Task 3.5:** Construct `Step3AnalyticsDashboard` including SVG Radar visualizer for RIASEC vector comparison and ordered match ranking cards.

Phase 4: Integration, Edge-Case Calibration & Offline LAN

Deployment Setup

- **Task 4.1:** End-to-end integration testing connecting React wizard frontend to Node.js local REST API.
- **Task 4.2:** Conduct offline execution verification: disconnect network interfaces, execute client-server flow over localhost and local intranet IP (192.168.x.x).
- **Task 4.3:** Package application host environment using a self-contained runtime wrapper (e.g., single executable Node.js package via pkg or standalone Docker container setup for LAN host machine).
- **Task 4.4:** Perform latency benchmark testing under simulated LAN loads (50 concurrent prospective student terminals).